

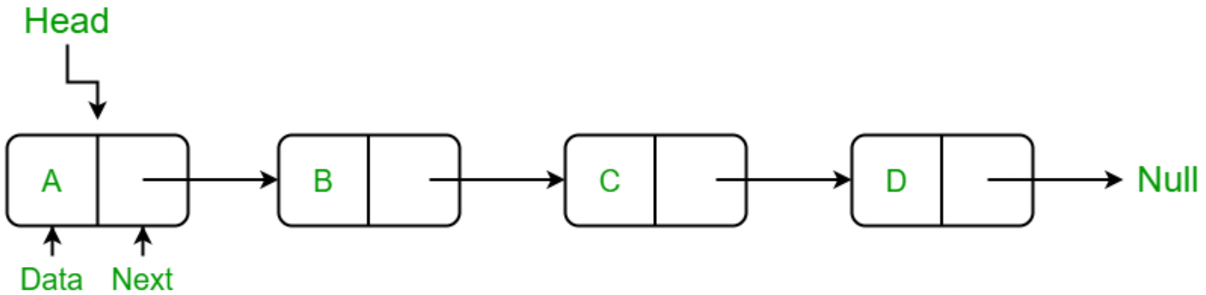
SEGi University

BCC1313 ALGORITHMS AND COMPLEXITY

Linked List Structures

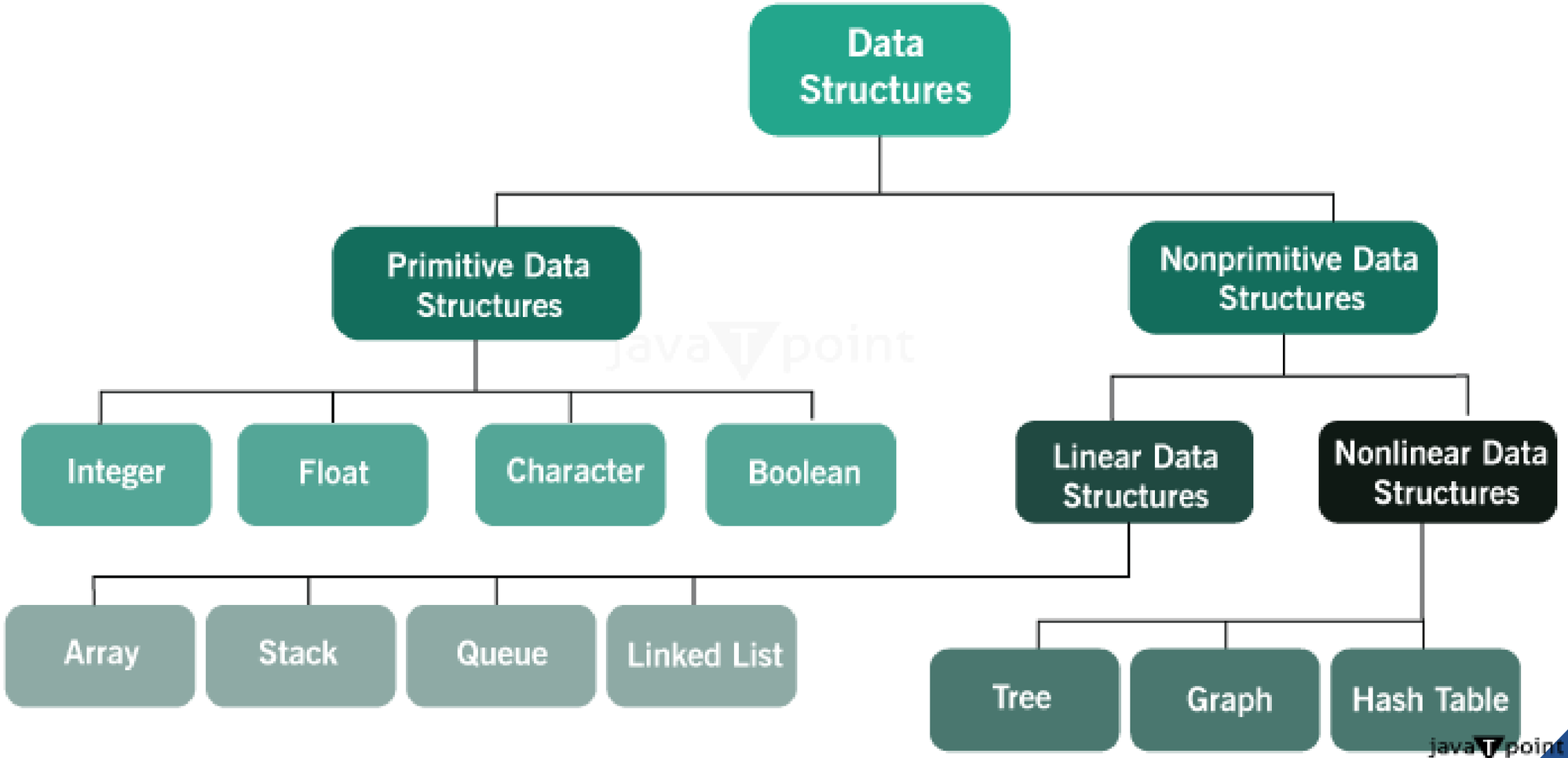
INTRODUCTION

- A **linked list** is a linear data structure where elements are stored in **nodes**, and each node points to the next one in the sequence.



- Linked lists are useful in situations where **dynamic memory and frequent changes** to the data are needed, as they allow for **easy insertions and deletions**.

TYPES OF DATA STRUCTURE



PRIMITIVE DATA STRUCTURE

- A **primitive data structure** is a basic data type that is directly operated upon by **machine-level instructions** (low-level command that CPU can directly execute) and serves as the foundation for more complex data structures.
- These data types are built into most programming languages and store single values. Example: int, float, double, char, bool.

NON-PRIMITIVE DATA STRUCTURE

- A **non-primitive data structure** is a complex data structure that is derived from **primitive data types**. These structures can store multiple values and are used to organize and manage data efficiently.
- Characteristics:
    a) Can store multiple values instead of a single value
    b) More complex than primitive data types
    c) Can be either **linear** or **non-linear**
    d) Requires more memory and processing power

LINEAR DATA STRUCTURE

- A **linear data structure** is a type of data structure where elements are arranged in a **sequential** manner, one after another.
- Characteristics:
  a) Elements are stored in a contiguous manner
  b) Can be traversed sequentially in a single run
  c) Memory utilization may not always be efficient

LINEAR DATA STRUCTURE

- Examples:
  a) **Array** – A fixed-size collection of elements stored at contiguous memory locations.
  b) **Linked List** – A collection of nodes where each node points to the next
  c) **Stack** – A LIFO (Last In, First Out) structure.
  d) **Queue** – A FIFO (First In, First Out) structure.

LINEAR DATA STRUCTURE

- Key characteristics:
  - **Single Sequence:** Elements are arranged in a straight line, making it easy to traverse from the first to the last element.
  - **Fixed or Dynamic Size:** Arrays have a fixed size, while linked lists can grow or shrink as needed.
  - **Simple Access Patterns:** Elements can be accessed easily, either by their index (in arrays) or by going through each element (in linked lists).

NON-LINEAR DATA STRUCTURE

- In a non-linear data structure, elements are not arranged sequentially, and each element may be connected to multiple elements.
- Characteristics:
  a) Elements are arranged hierarchically or in a complex relationship
  b) Efficient memory utilization in certain cases
  c) Traversal can be complex, requiring recursive or special algorithms

NON-LINEAR DATA STRUCTURE

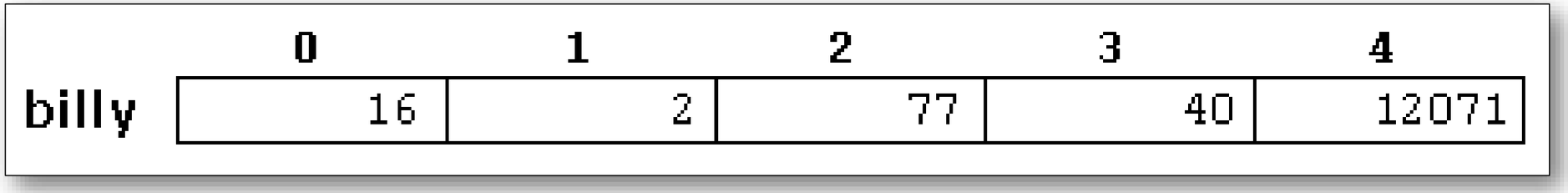
- Examples:
  a) **Tree** – A hierarchical structure where each node has a parent-child relationship (e.g., Binary Tree, BST, AVL Tree)
  b) **Graph** – A collection of nodes (vertices) connected by edges, used to represent networks.
  c) **Heap** – A specialized tree-based structure used for priority queues.

LINKED LIST vs ARRAY

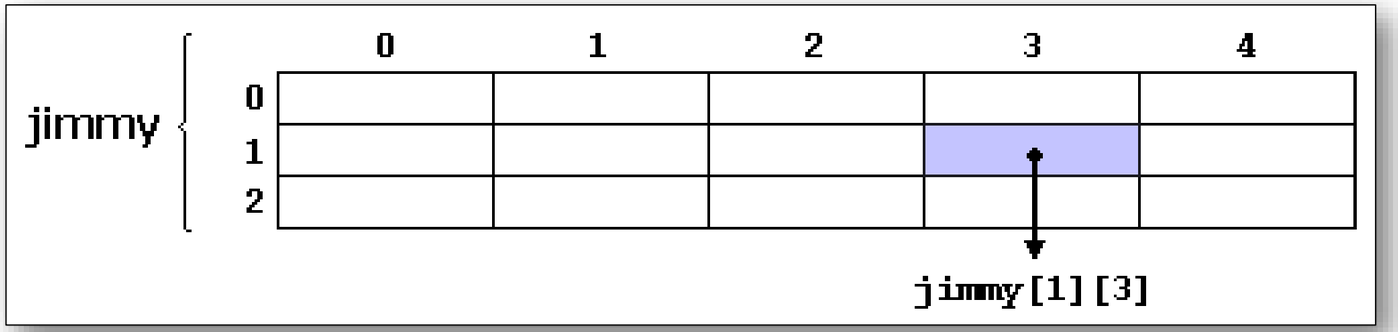
- Both linked lists and arrays are used to store collections of elements.
- Arrays store elements in **contiguous memory locations** and have a **fixed size**, meaning the number of elements must be predetermined.
- This allows for fast, direct access to elements using an **index**, but resizing an array can be inefficient since it involves copying all elements to a new location.

```
int billy [5];
```

```
int billy [5] = { 16, 2, 77, 40, 12071 };
```

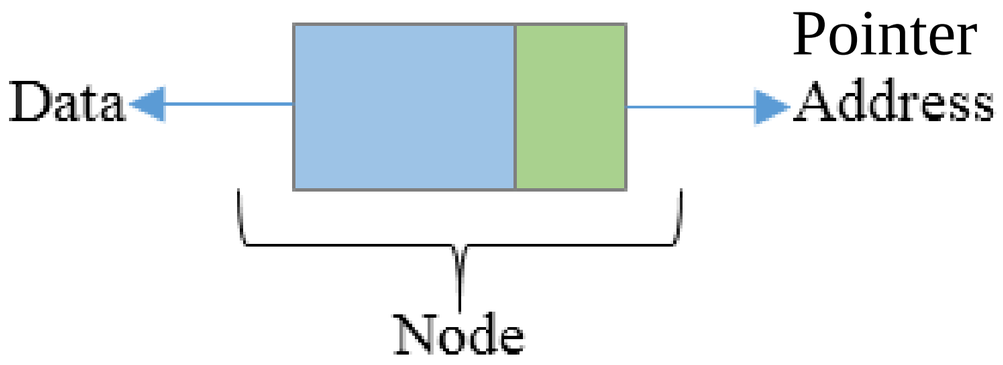


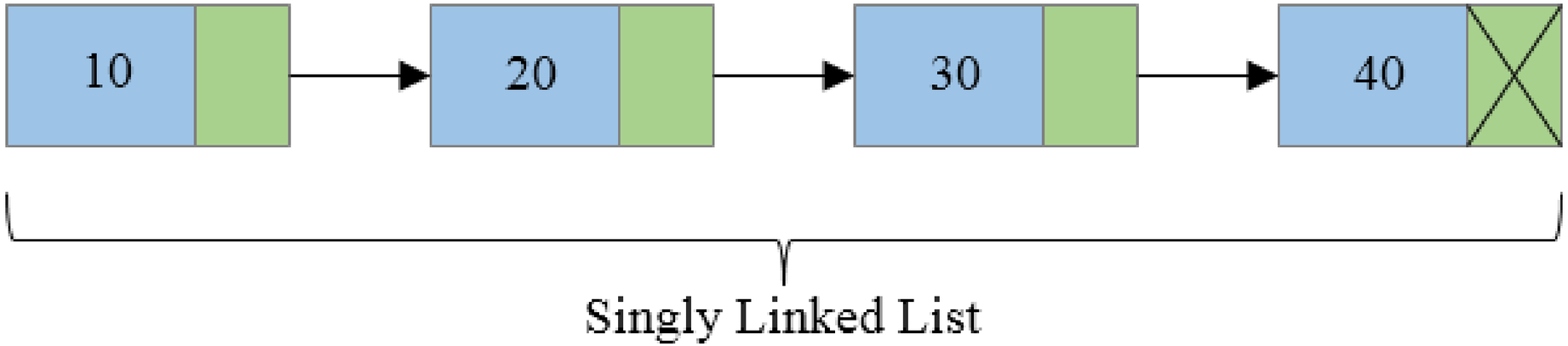

```
int jimmy [3][5];
```



LINKED LIST vs ARRAY

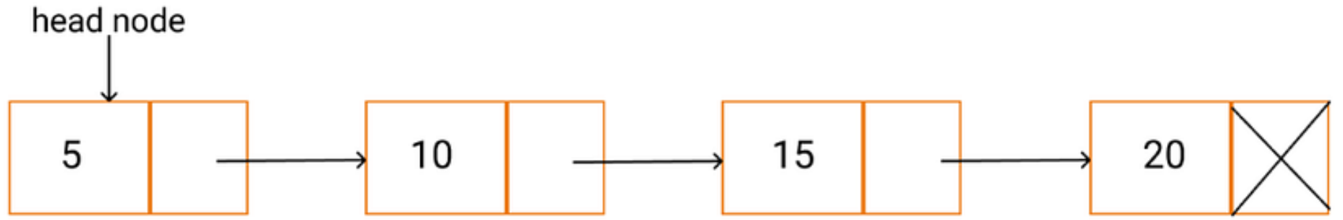
- In contrast, linked lists use a **node-based structure** where each node contains **data** and a **pointer** to the next node.
- This flexible design allows linked lists to **dynamically grow or shrink** in size without the need for reallocating memory.
- However, linked lists do not allow direct access to elements, requiring **traversal** from the beginning to find a specific node.


Data
Pointer
Address
Node

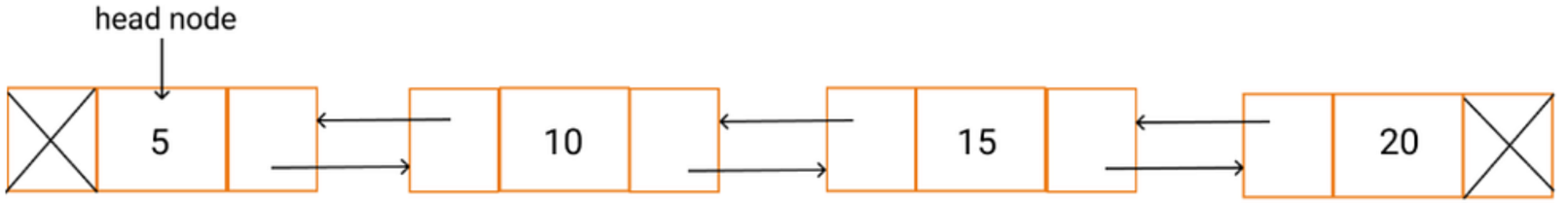

10
20
30
40
Singly Linked List

Types of Linked List

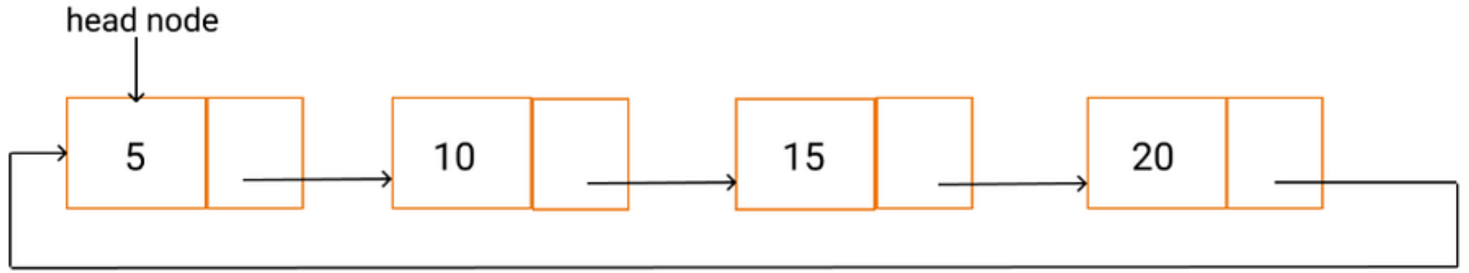
Singly Linked List

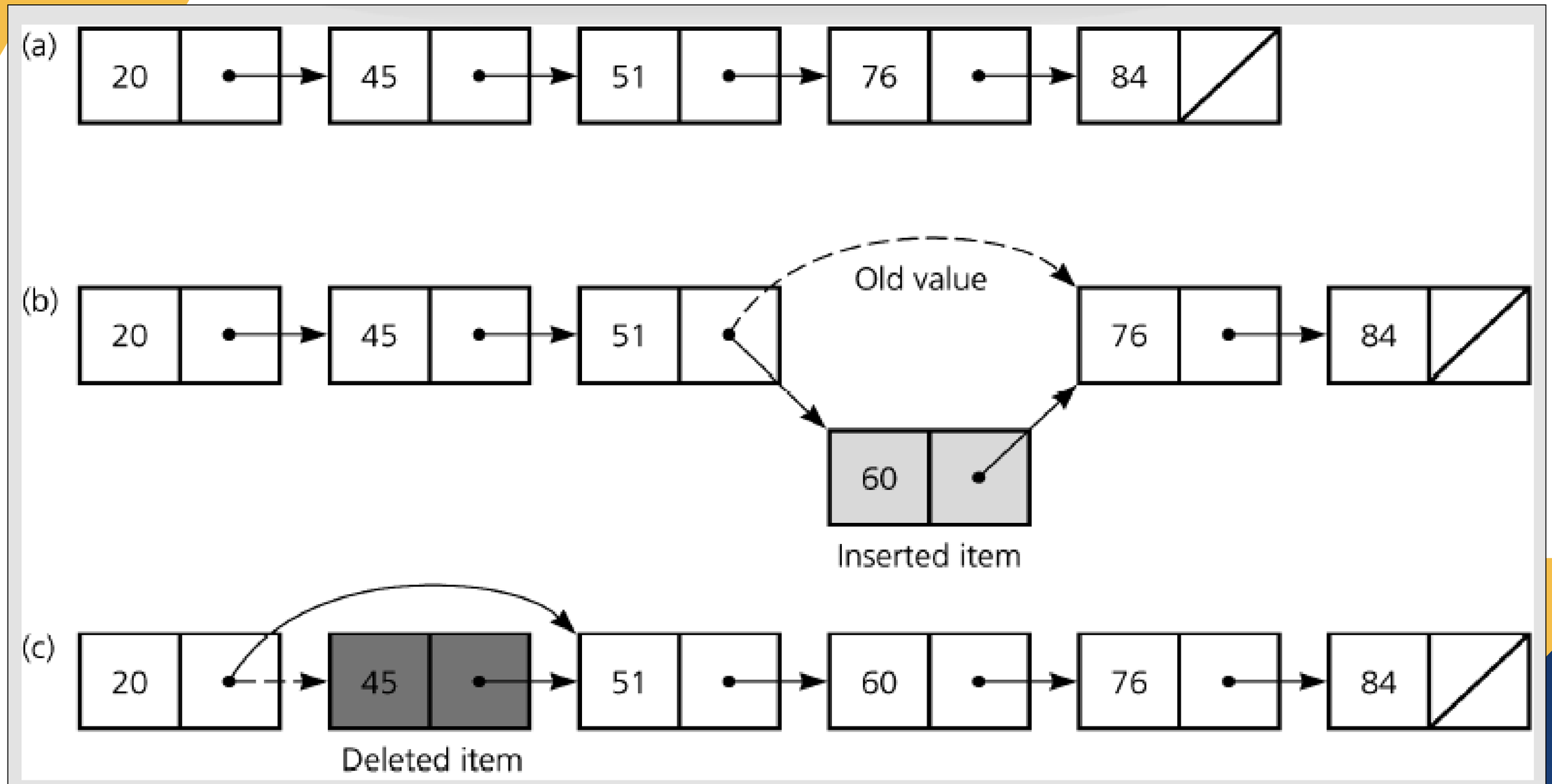


Doubly Linked List



Circular Linked List

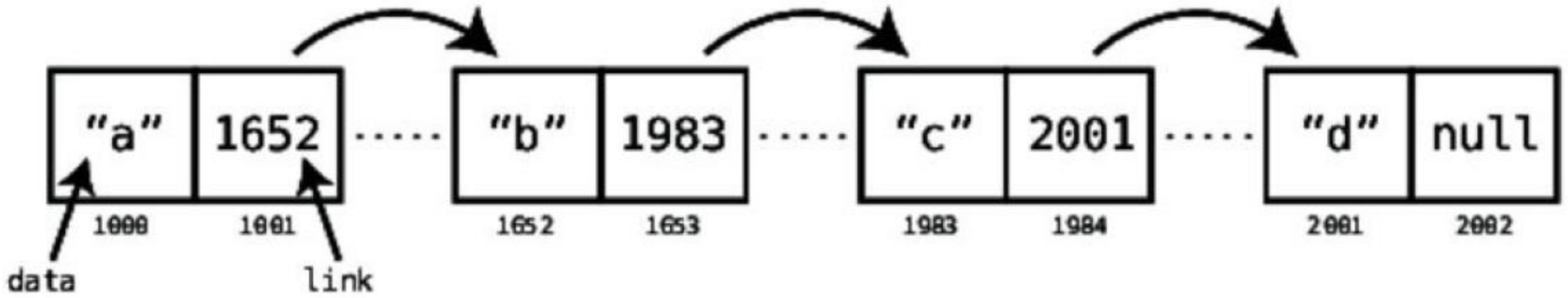




Insertion or deletion with pointer modifications.

LINKED LIST STRUCTURE

- A linked list consists of a sequence of nodes, each containing data and a reference to the next node.
- Every node stores the **memory address** of the next node, which is also referred to as a **link**.

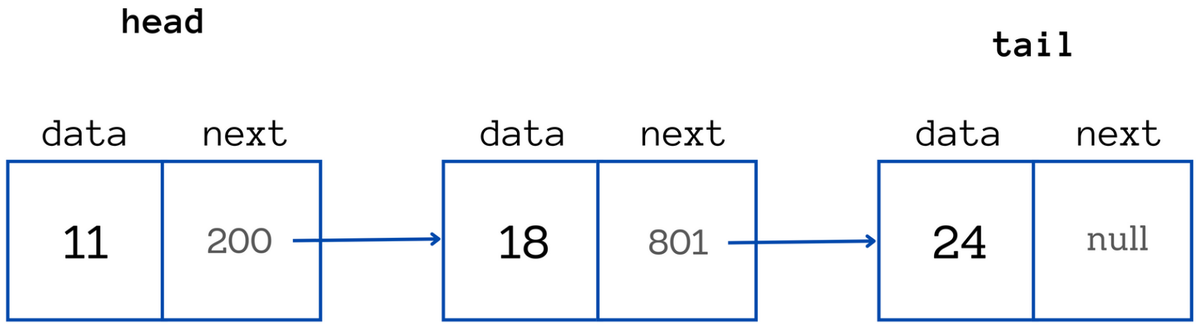


LINKED LIST STRUCTURE

- Even though the nodes are not in a contiguous memory, the **nodes are stored linearly through links**.
- Every node has the **address** of its succeeding node. That is how each node can access its succeeding node.

LINKED LIST STRUCTURE

- The first node of the linked list is called the **head node**. It is the **starting point** of a linked list.
- The last node is called the **tail node**. As there is no node after the last node, the last node always points to the **null**.
- A null pointer does **not point to any memory location**.



IMPLEMENTING A LINKED LIST

- **Create a Node**

```
class Node {
    public:
        int data;
        Node* next;

    Node(int data) {
        this->data = data;
        next = nullptr;
    }
};
```

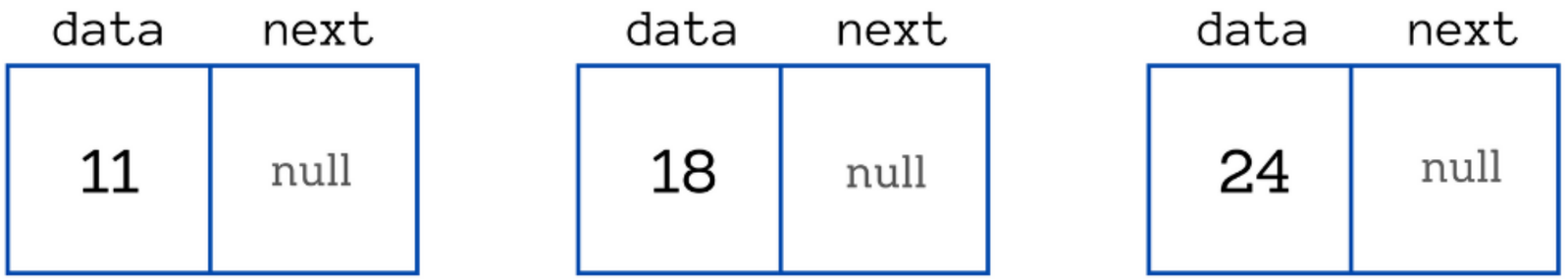
The Node class represents a node with two instance variables: data (holds the data), and next (holds a reference to the next node).

The constructor takes an int argument data to initialize the data variable and sets the next variable to null by default.

IMPLEMENTING A LINKED LIST

- **Create a Node**

```
Node* node1 = new Node(11);
Node* node2 = new Node(18);
Node* node3 = new Node(24);
```



IMPLEMENTING A LINKED LIST

- **Link the Nodes**

```
class LinkedList {
public:
    Node* head;

    LinkedList() {
        head = nullptr;
    }
};
```

The head member is declared as Node* head, allowing it to point to the first node in the linked list or be nullptr if the list is empty.

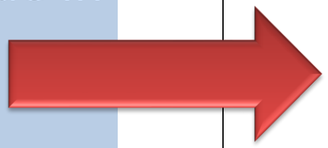
The constructor initializes head to nullptr, indicating that the list starts empty.

IMPLEMENTING A LINKED LIST

- **Link the Nodes**

Setting the head node to the first node in the list.

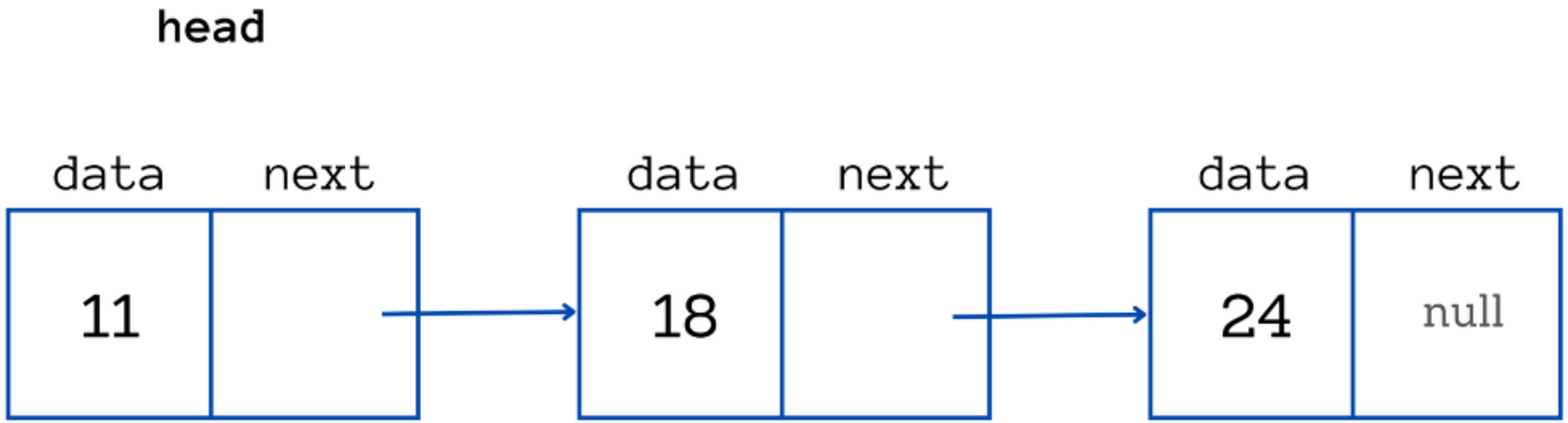
The node1 point to node2, and the next of node2 point to node3



```
int main()
{
    LinkedList list;
    Node* node1 = new Node(11);
    Node* node2 = new Node(18);
    Node* node3 = new Node(24);
    list.head = node1;
    node1->next = node2;
    node2->next = node3;
    return 0;
}
```

IMPLEMENTING A LINKED LIST

- **Link the Nodes**



IMPLEMENTING A LINKED LIST

- **Append a Nodes**
  - Appending a node means adding a node to the end of a linked list.
  - There are two cases to consider when appending a node:
    1. Appending to an **empty linked list.**
    2. Appending to a **non-empty linked list.**

IMPLEMENTING A LINKED LIST

- **Append a Nodes to an Empty Linked List**
  - To append a node to an empty linked list, need to make sure the linked list is empty by **checking if the head node is null**.
  - If the head node is null, simply set head to the new node.

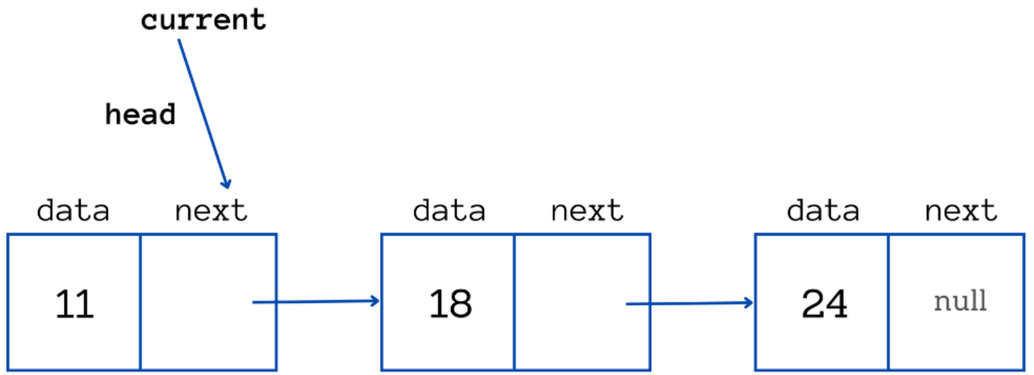
```
if (head == nullptr) {
    head = newNode;
}
```

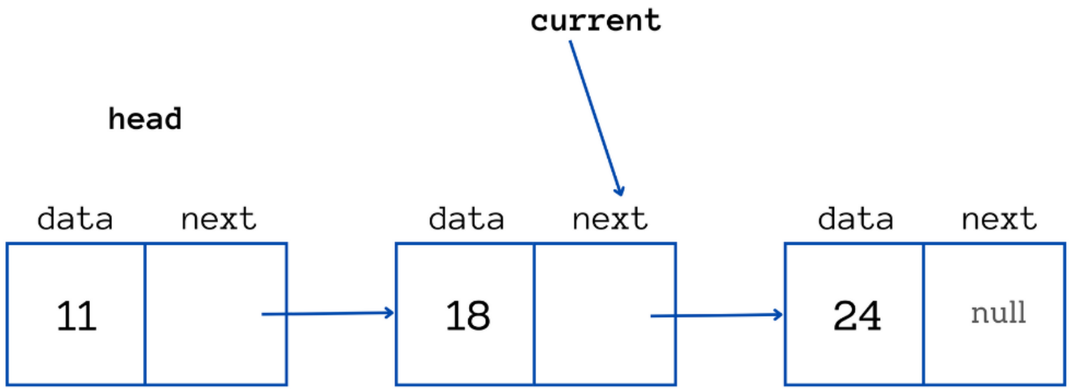
IMPLEMENTING A LINKED LIST

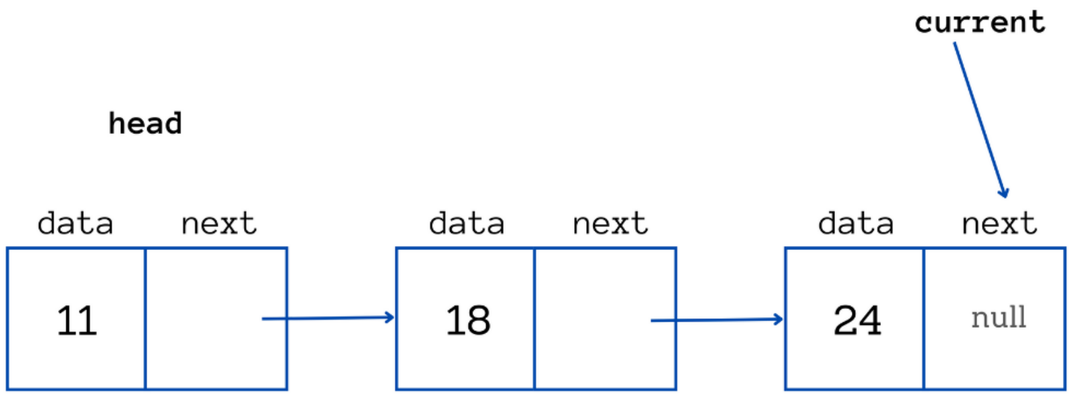
- **Append a Nodes to Non-Empty Linked List**
  - If there are one or more nodes in a linked list, it is a non-empty linked list.
  - To append a node to a non-empty linked list, **link the last node to the new node.**

IMPLEMENTING A LINKED LIST

- **Append a Nodes to Non-Empty Linked List**
  - Elements in a linked list cannot be access directly. Thus, need to traverse from the head node to the last node.
  - To do that, create a **temporary pointer** (e.g. current pointer) that points to the head node.
  - Make current point to its next node, till the next of the current node points to null.

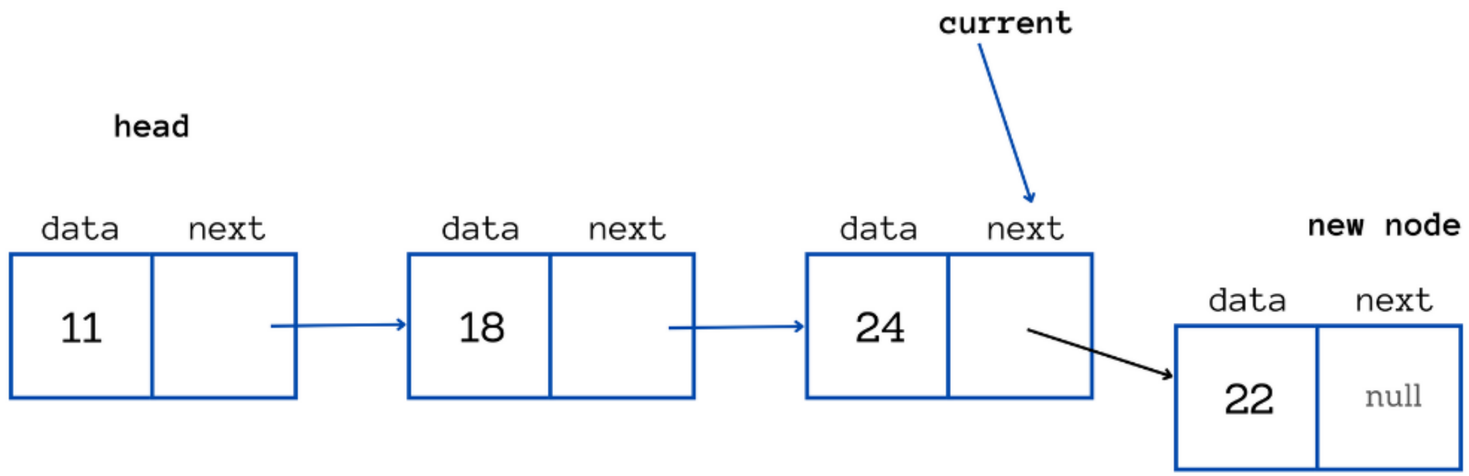

current
head
data
next
11
data
next
18
data
next
24
null


current
head
data
next
11
data
next
18
data
next
24
null


current
head
data
next
11
data
next
18
data
next
24
null

IMPLEMENTING A LINKED LIST

- **Append a Nodes to Non-Empty Linked List**
  - When the next node of current is null, you can then make the next of the current node point to the new node.





```
void append(int data) {
    Node* newNode = new Node(data);
    if (head == nullptr) {
        head = newNode;
    }
    else {
        Node* current = head;
        while (current->next != nullptr) {
            current = current->next; // Traverse to the last node
        }
        current->next = newNode; // Link the new node to the last node
    }
}
```

IMPLEMENTING A LINKED LIST

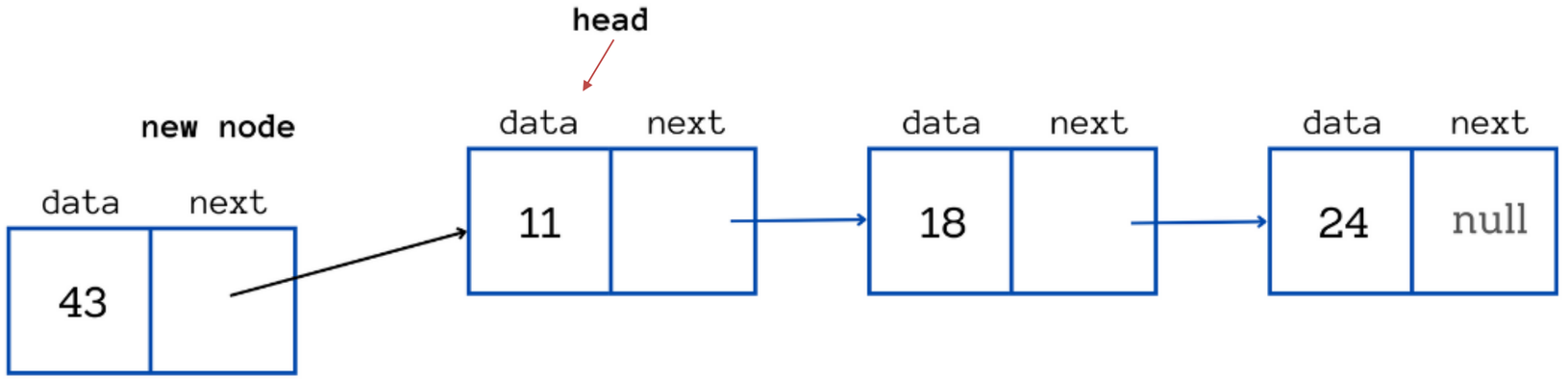
- **Insert a Nodes**
  - Inserting a node means adding a node to a given index.
  - There are two cases to consider when inserting a node:
    1. Inserting a node at the first index.
    2. Inserting a node at a given index.

IMPLEMENTING A LINKED LIST

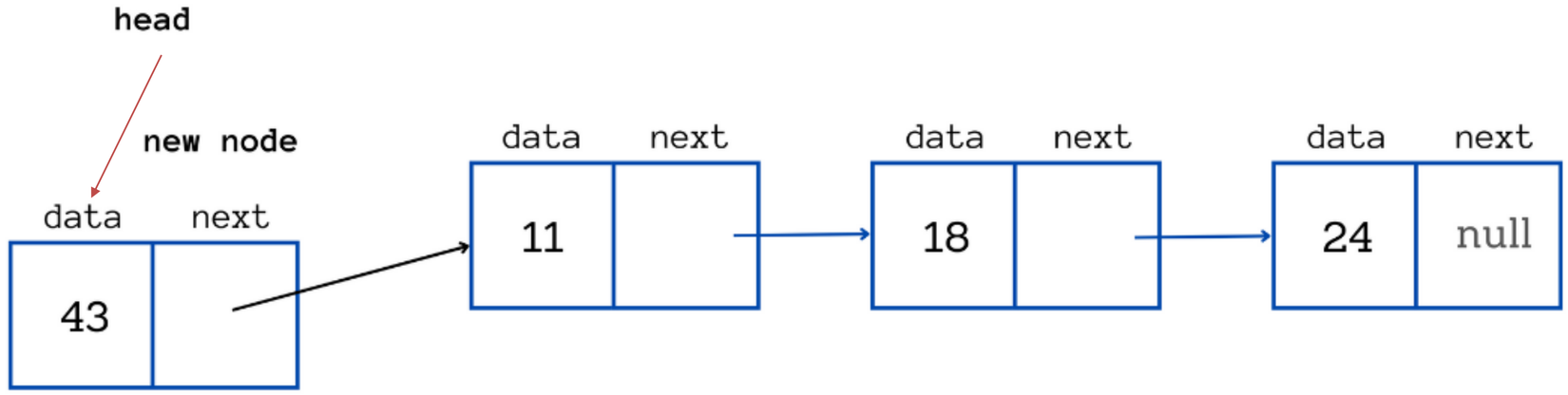
- **Insert a Nodes at the First Index**
  - To insert a node at the first index, make next of the new node point to the head node
  - Then, set the head to the new node.

```
void insertAtHead(int data) {
    Node* newNode = new Node(data); // Create a new node
    newNode->next = head;           // Set newNode's next to current head
    head = newNode;                 // Update head to point to the new node
}
```

1



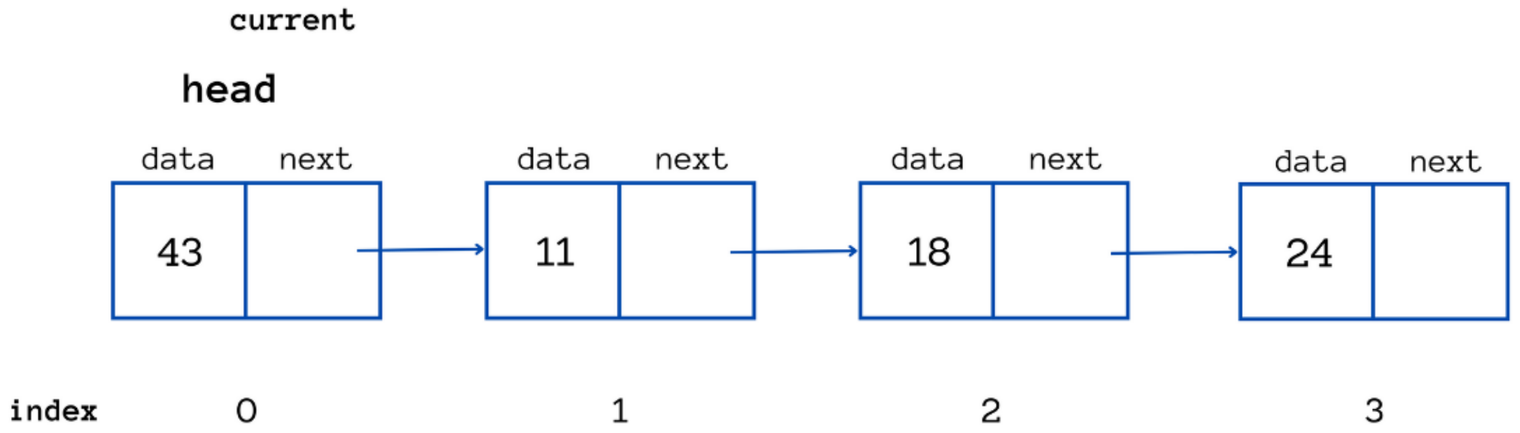
2



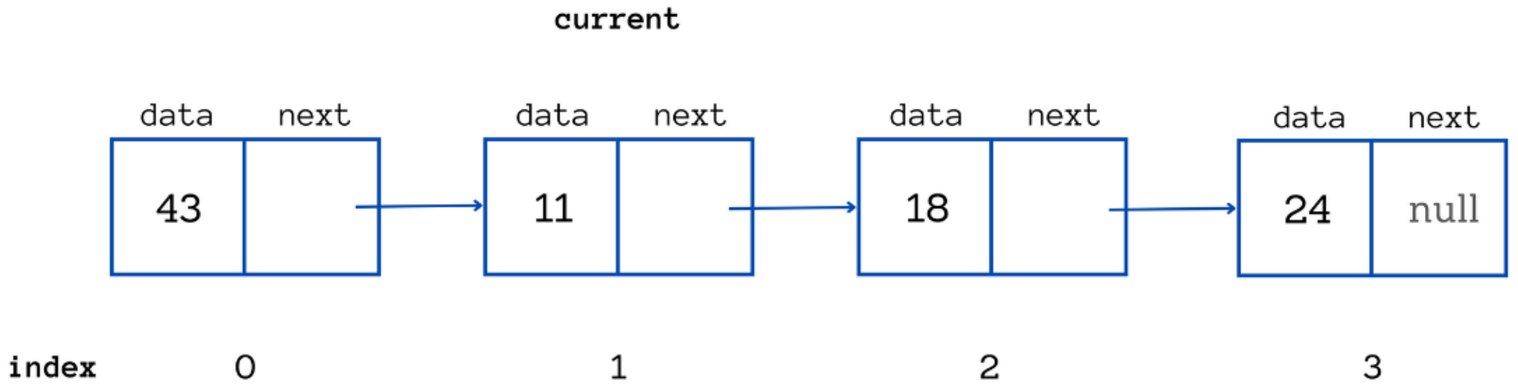
IMPLEMENTING A LINKED LIST

- **Insert a Nodes at any Position**
  - Suppose to add a node at index 2 in the linked list, traverse the node that comes before index 2.
  - Then, create a new node and make the next of the new node point to the next of the current node.
  - Finally, make the next of current point to the new node.

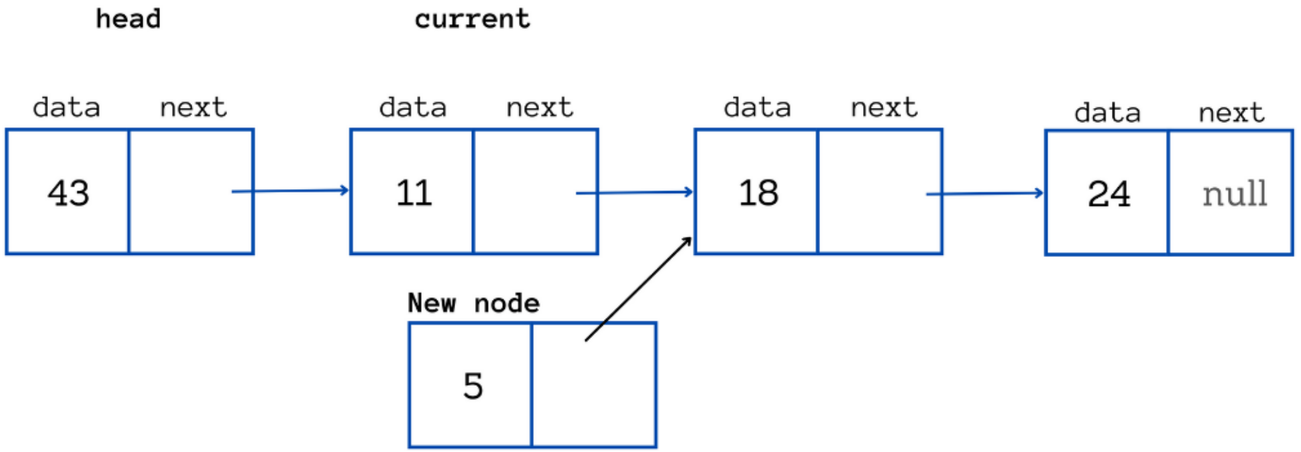
1


current
head
data
next
43
11
18
24
index
0
1
2
3

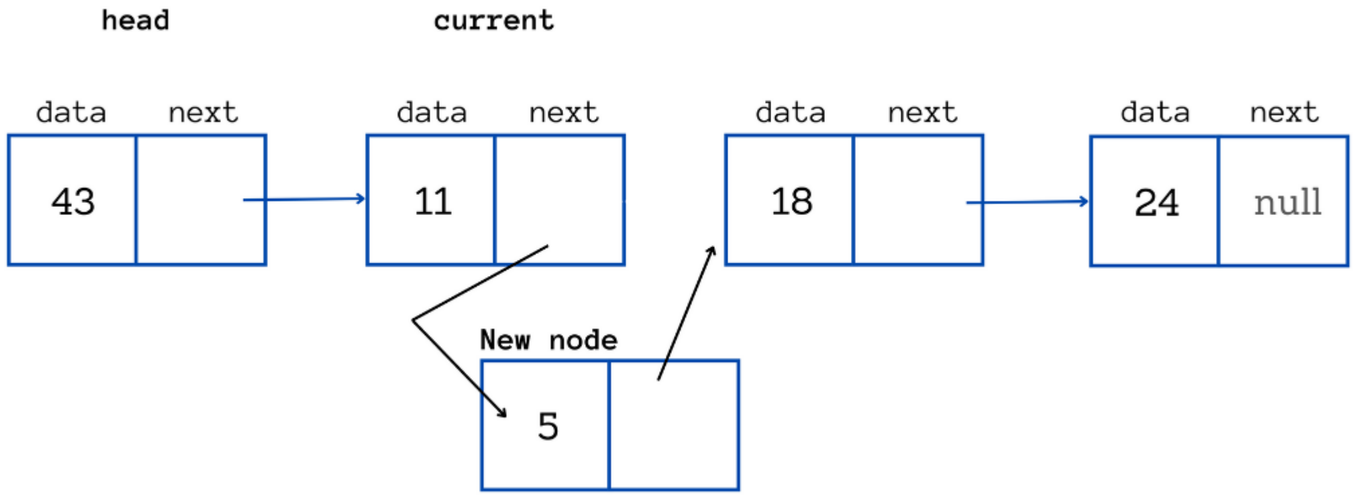
2


current
data
next
43
11
18
24
null
index
0
1
2
3

3


head
current
data
next
43
11
18
24
null
New node
5

4


head
current
data
next
43
11
18
24
null
New node
5

```
void insertAtIndex(int data, int index) {
    Node* newNode = new Node(data);
    if (index == 0) {
        newNode->next = head;
        head = newNode;
        return;
    }
    Node* current = head;
    for (int i = 0; i < index - 1 && current != nullptr; i++) {
        current = current->next;
    }
    if (current != nullptr) {
        newNode->next = current->next;
        current->next = newNode;
    } else {
        cout << "Index out of bounds!" << endl;
    }
}
```

IMPLEMENTING A LINKED LIST

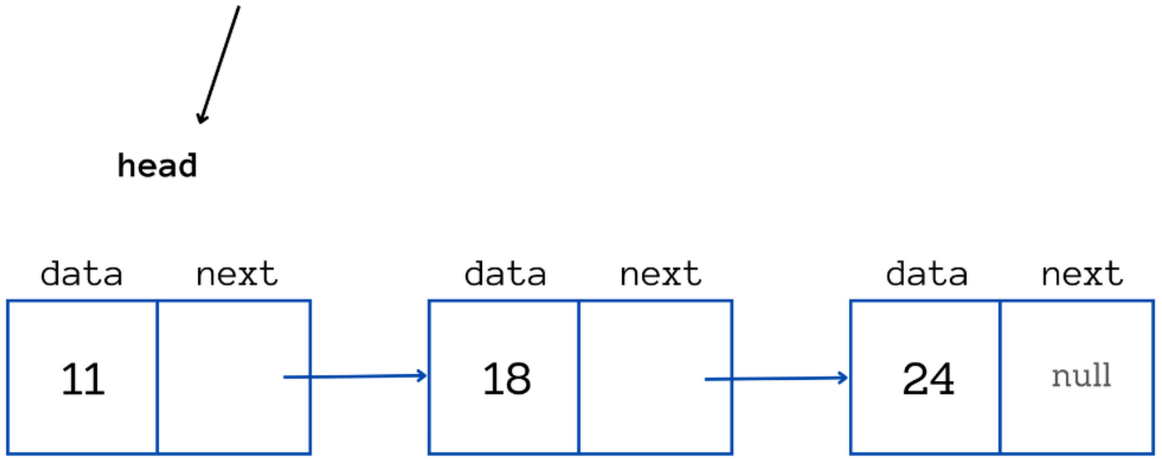
- **Delete a Node**
  - There are two ways to delete nodes in a linked list:
    1. Deleting the head node.
    2. Deleting a node at a given position.

IMPLEMENTING A LINKED LIST

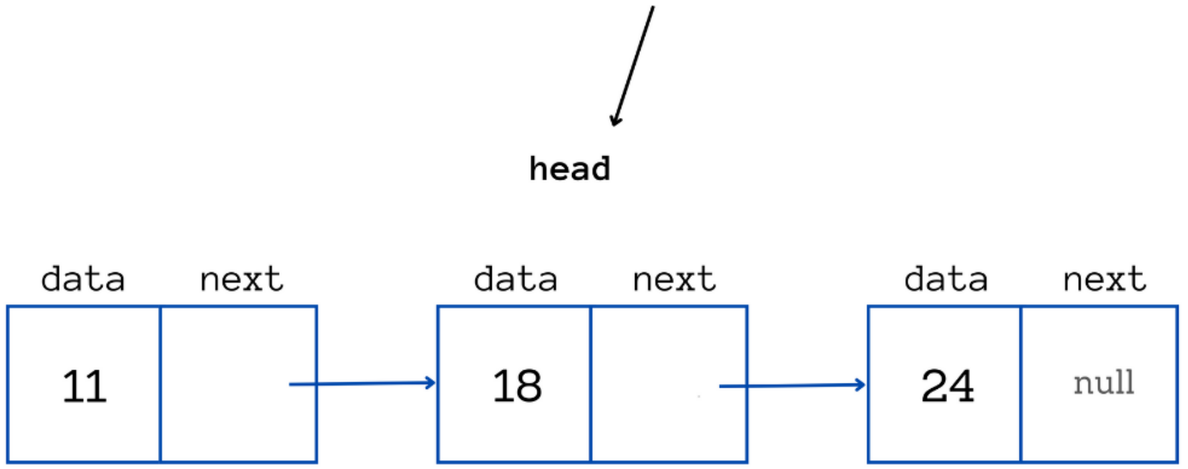
- **Delete a Head Node**
  - Deleting the head node is simple by store the data of the head node in a temporary variable.
  - Then, set the head pointer to point to the next node after the head node.

```
if (index == 0) {
        temp = head;                // Temporary pointer to the head
        deletedValue = head->data; // Store the value to return
        head = head->next;          // Move head to the next node
        delete temp;                // Delete the old head node
        return deletedValue;
    }
```

1


head
data
next
11
data
next
18
data
next
24
null

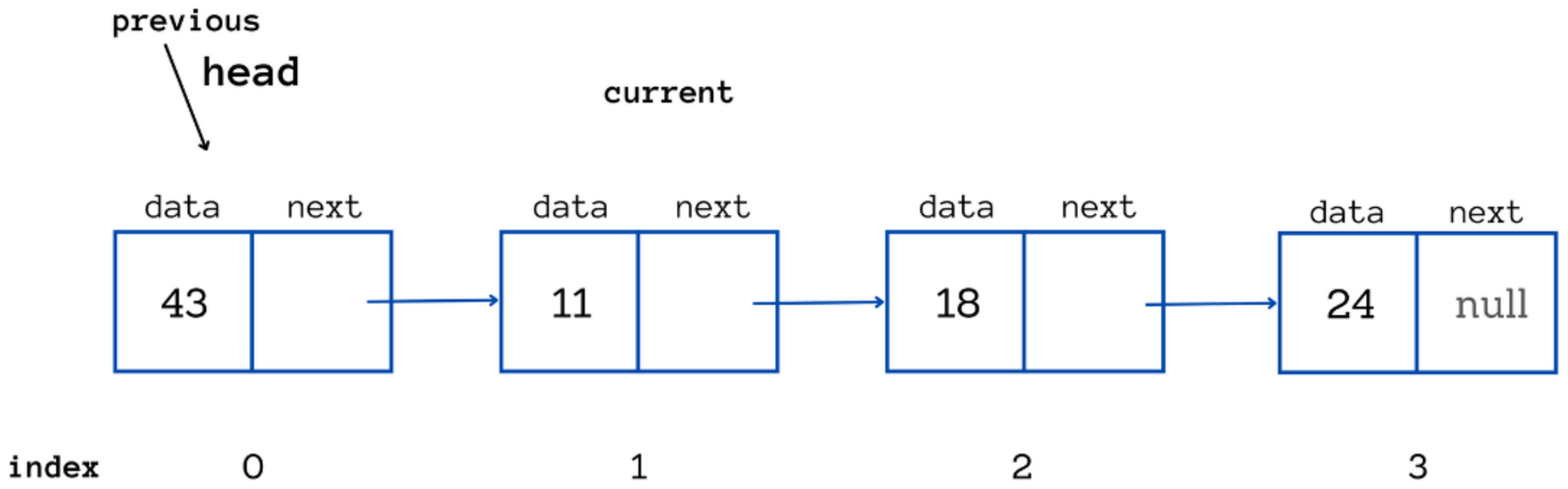
2


head
data
next
11
data
next
18
data
next
24
null

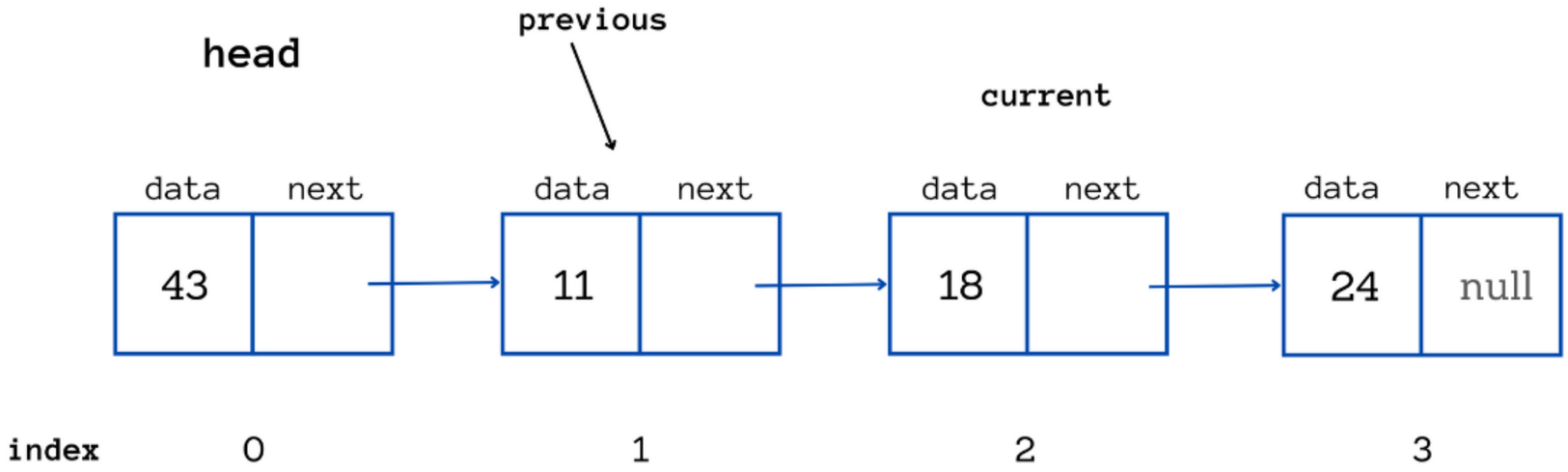
IMPLEMENTING A LINKED LIST

- **Delete a Node at a Given Position**
  - Suppose to delete a node at index 2, access the node to delete and the node before it.
  - Set two temporary pointers (e.g. pointers previous and current).
  - Let the previous point to null and current point to the head node.
  - Make the next of previous point to the next of the current node.

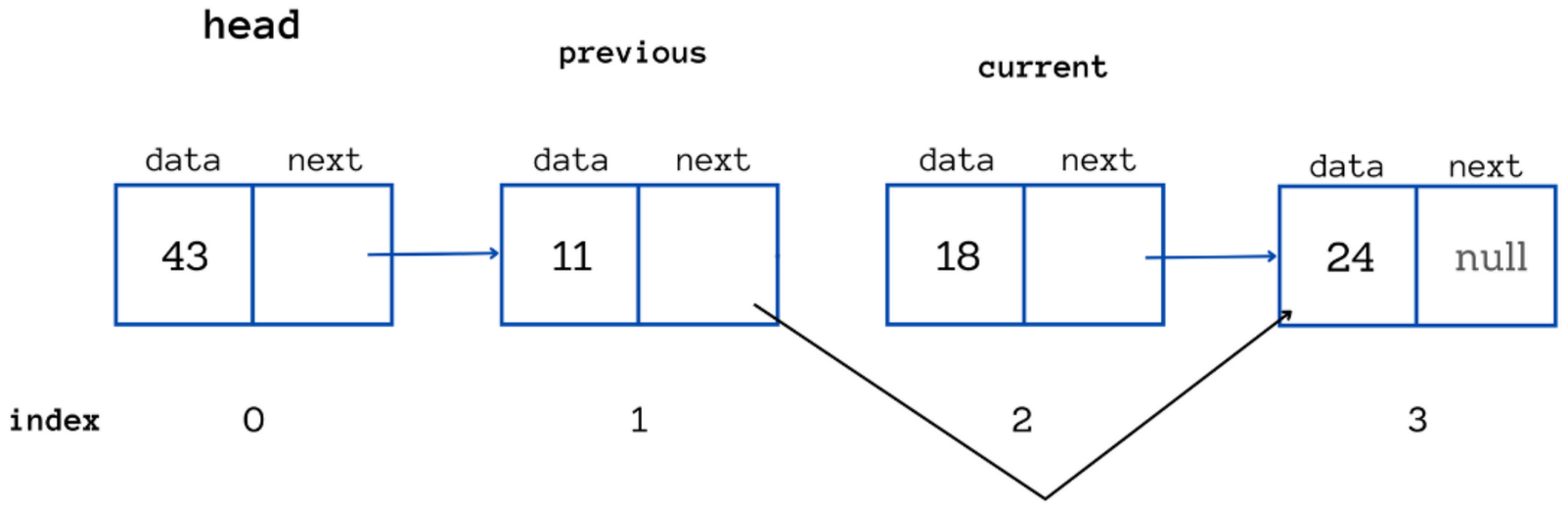
1


previous
head
current
data
next
43
11
18
24
null
index
0
1
2
3

2


head
previous
current
data
next
43
11
18
24
null
index
0
1
2
3

3



- After removing the link to the node at index 2, it **is no longer accessible** through any reference in the linked list.
- The removed node will be automatically handled by the **garbage collector** when it is no longer reachable through any references.

```
int deleteAtIndex(int index) {
    if (head == nullptr) {
        cout << "List is empty." << endl;
        return -1;
    }
    Node* temp;
    int deletedValue;
    if (index == 0) {
        temp = head;
        deletedValue = head->data;
        head = head->next;
        delete temp;
        return deletedValue;
    }
    Node* current = head;
    for (int i = 0; i < index - 1 && current->next != nullptr; i++) {
        current = current->next;
    }
    if (current->next == nullptr) {
        cout << "Index out of bounds." << endl;
        return -1;
    }
    temp = current->next;
    deletedValue = temp->data;
    current->next = temp->next;
    delete temp;
    return deletedValue;
}
```

```
void display() {
    Node* current = head;
    while (current != nullptr) {
        cout << current->data << " -> ";
        current = current->next;
    }
    cout << "nullptr" << endl;
}
```

```
int main()
{
    LinkedList list;

    list.append(10);
    list.append(20);
    list.append(30);
    list.append(40);
    cout << "List after appending nodes: ";
    list.display();

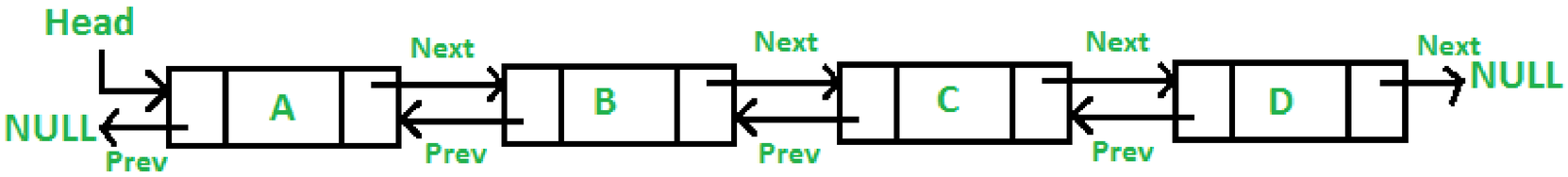
    // Insert a node at index 2
    list.insertAtIndex(25, 2);
    cout << "List after inserting 25 at index 2: ";
    list.display();  // Output: 10 -> 20 -> 25 -> 30 -> 40 -> nullptr

    // Delete a node at index 3
    int deletedValue = list.deleteAtIndex(3);
    cout << "Deleted value: " << deletedValue << endl;
    cout << "List after deleting node at index 3: ";
    list.display();  // Output: 10 -> 20 -> 25 -> 40 -> nullptr

    return 0;
}
```

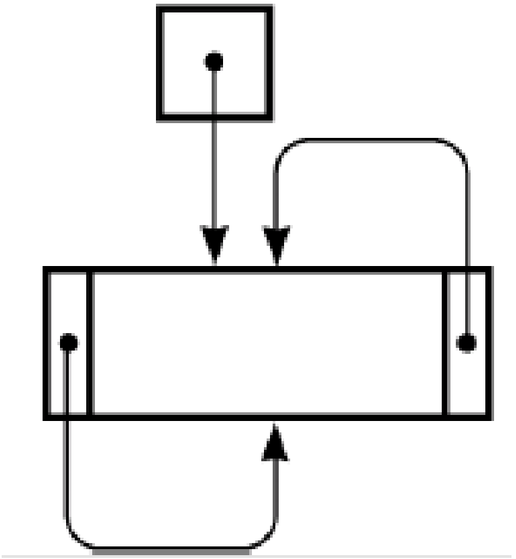
DOUBLY LINKED LIST

- A **doubly linked list** is like a linked list, except that each node has **two links**: one that points to the **next node**, and one that points to the **preceding node**.
- In addition, the doubly linked list keeps track of both the first and last nodes.



DOUBLY LINKED LIST

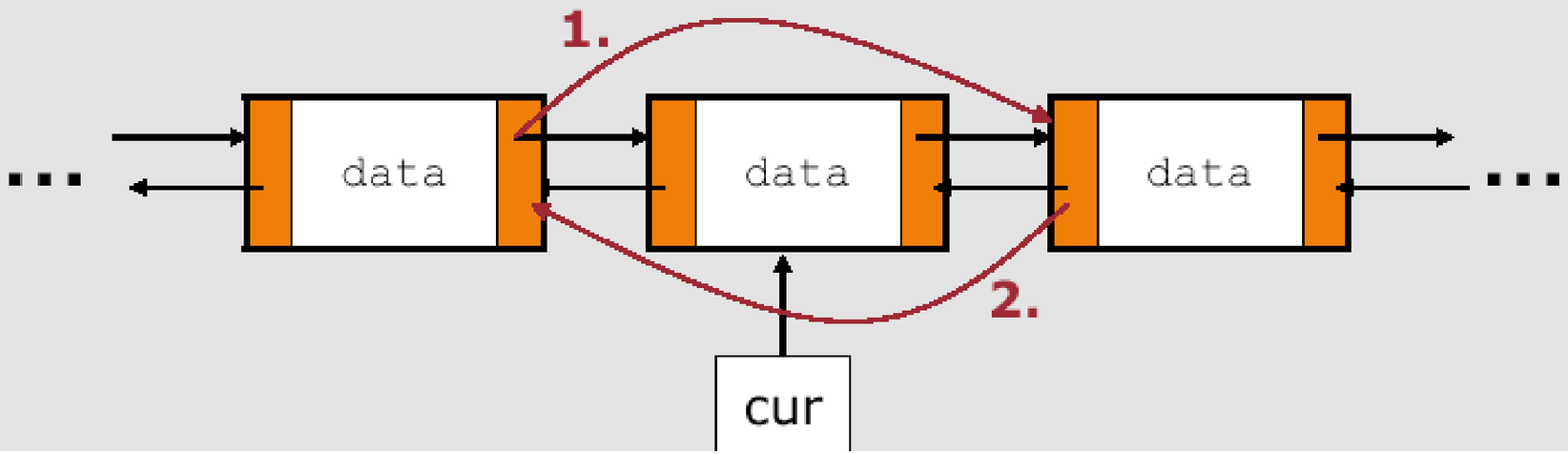
- An empty circular doubly linked list:



DOUBLY LINKED LIST

- To delete the node to which ***cur*** points:

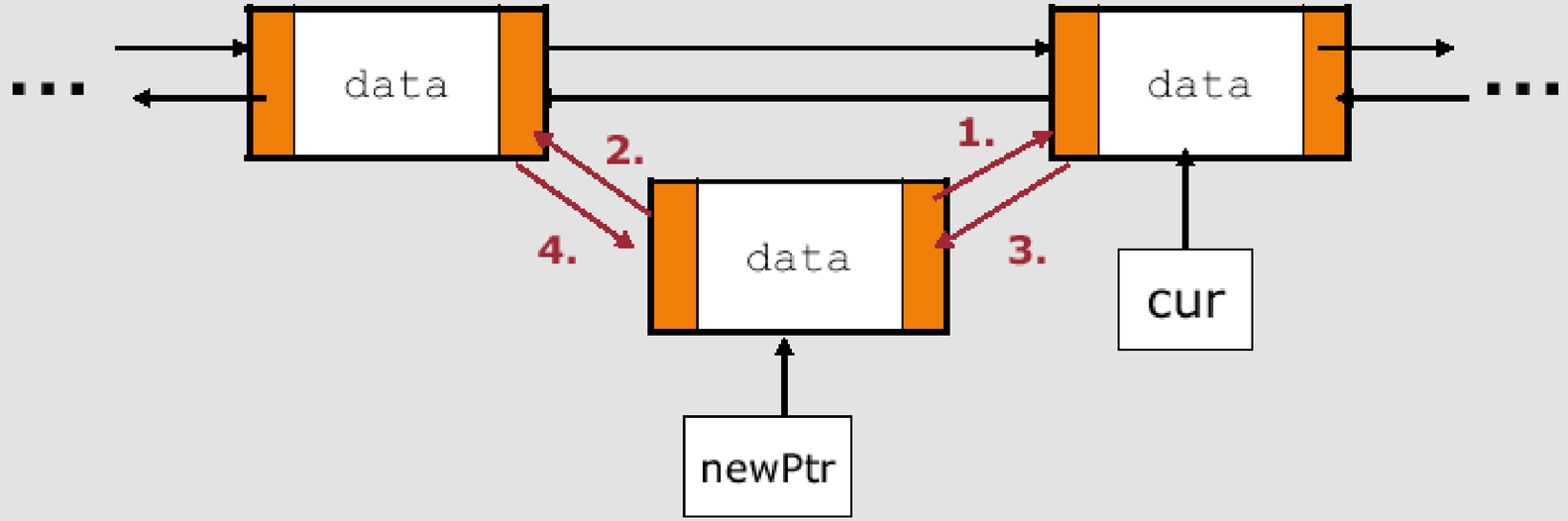
```
1. (cur->precede)->next = cur->next;
2. (cur->next)->precede = cur->precede;
```



DOUBLY LINKED LIST

- To insert a new pointed to by ***newPtr*** before the node pointed to by cur:

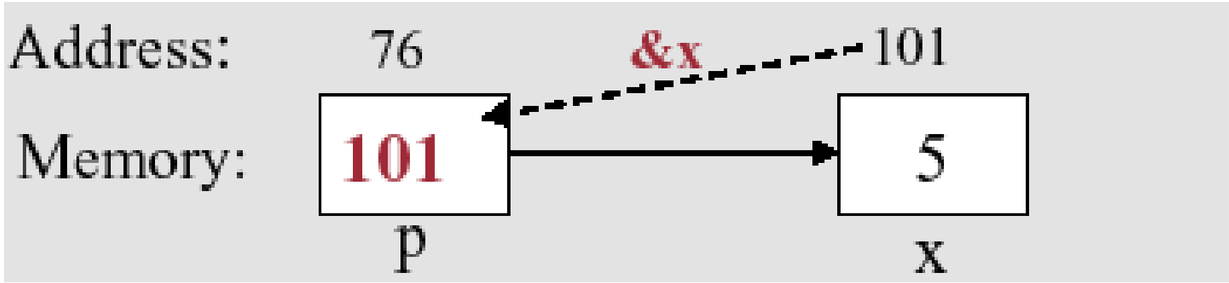
```
1. newPtr->next = cur;
2. newPtr->precede = cur->precede;
3. cur->precede = newPtr;
4. newPtr->precede->next = newPtr;
```



ADT List

- To place the address of a variable into a pointer variable, we can use the address-of operator (&):

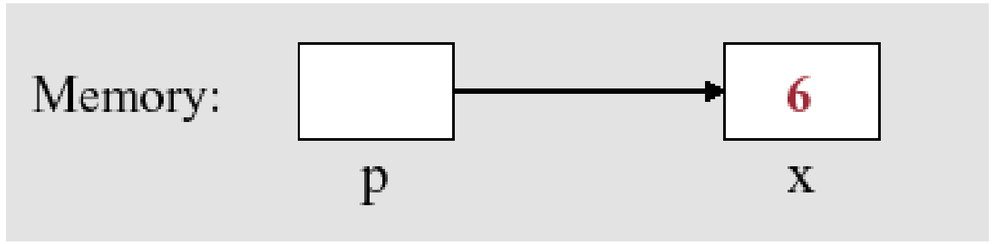
```
p = &x;
```



- The expression *p represents the memory cell to which p points.

```
*p = 6;
```

```
cout<<*p;
```



ADT List

- Compare the following statements:

```
int *p, q;
int *p, *q;
```

p is a pointer variable, q is an integer variable.

p and q are all pointer variables.

- When declare:

```
int *p, *q;
int x;
```

  - The memory for the pointer variables ***p*** and ***q,*** and for the integer variable ***x*** is allocated statically.

ADT List

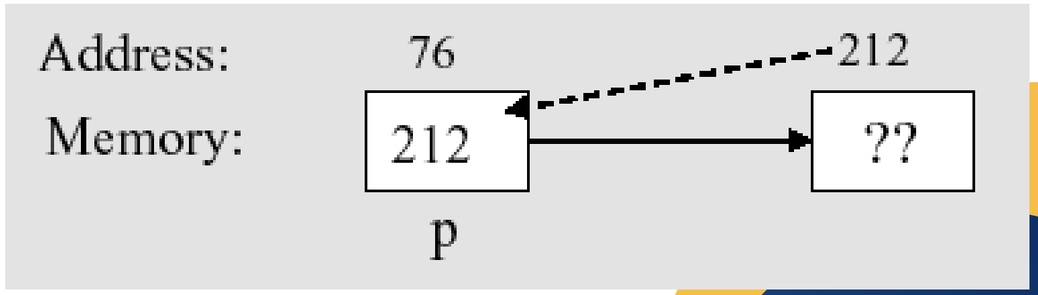
- How about this statements:

```
p = x;
```

  - The assignment is illegal!
  - Type clash: `x` is an integer variable, while `p` is a pointer variable.

ADT List

- Memory allocation can occur at **execution time** i.e. dynamic allocated variable. This gives flexibility to the program to request and release memory as needed.
- Allocate memory using the **new** operator:




```
int *p;
```

```
p = new int;
```

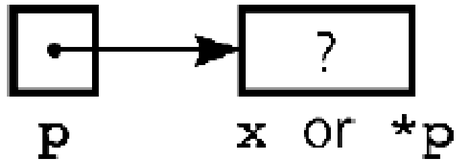
  - The initial content of this new cell is undetermined.
  - Returns a pointer (or memory address) to this new cell.

ADT List

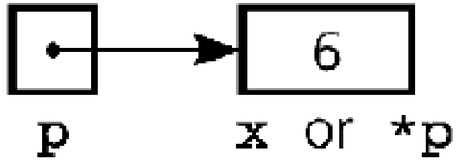
(a)
```
int *p, *q;
int      x;
```



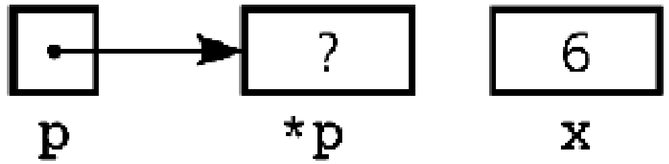
(b)
```
p = &x;
```



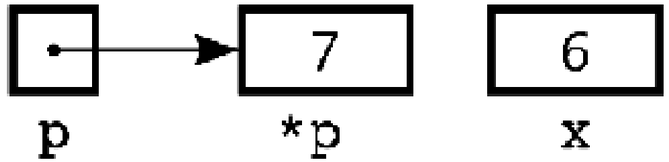
(c)
```
*p = 6;
```



(d)
```
p = new int;
```



(e)
```
*p = 7;
```



ADT List

- What is wrong with the following statements?

```
int *p, *q;

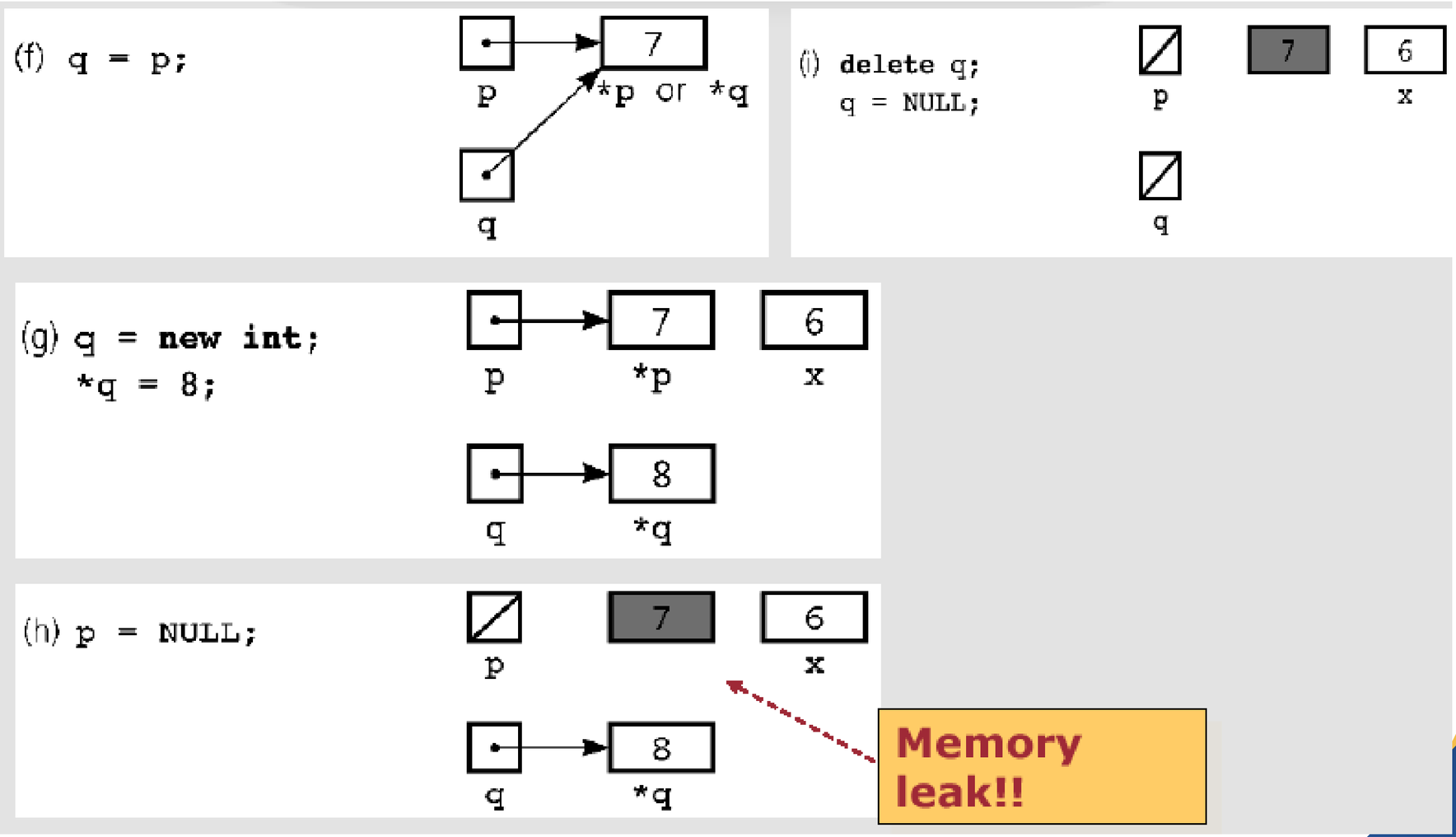
p = new int;

q = new int;

p = q;
```

- *p* points to the same cell as *q*.
- The cell *p* formerly pointed to is lost; it cannot be referenced.

ADT List


(f) q = p;
7
p
*p or *q
q
(i) delete q;
q = NULL;
p
7
6
x
q
(g) q = new int;
*q = 8;
7
6
p
*p
x
8
q
*q
(h) p = NULL;
p
7
6
x
8
q
*q
Memory leak!!

SUMMARY

- A linked list is a linear data structure where elements are stored in **nodes**: **data** and **pointer** (link).
- A linked lists are **dynamic** and can grow or shrink in size during runtime, making them efficient for operations such as insertion and deletion.
- Common operations:
  - **Insertion**: Add a new node at the beginning, end, or a specific index.
  - **Deletion**: Remove a node from the list.
  - **Traversal**: Access each node in the list sequentially

SUMMARY

- **Applications**:
  - Music playlists
  - Image viewer
  - Undo/redo in software
  - Web browser navigation
  - OS task scheduling
  - Implementing graph, queus, and stack

Practice

1. A pointer variable contains the _____ of a memory cell.
    a) content
    b) address
    c) data type
    d) ADT

Practice

2. The C++ address-of operator is _____.

   a) <<

   b) ->

   c) &

   d) *

Practice

3. In a linear linked list, _____.

   a) the next pointer of each node has the value NULL

   b) the last node points to the first node

   c) the precede pointer of the dummy head node points to the last node

   d) the next pointer of the last node has the value NULL

Practice

- Tutorial 3
- Lab 3